

A scenic mountain landscape with green hills, rocky cliffs, and a cloudy sky. The foreground shows a rocky, grassy slope. In the middle ground, there are steep, rocky cliffs and green hills. The background features more distant, hazy mountain ranges under a sky filled with white and grey clouds.

AI & Software Engineering Workshop

Week 1 — Day 5: Deploy and Ship

Edik Simonian, Summer 2026

From polling to webhook

- All week: **polling** — your laptop asks Telegram for updates
- Production: **webhook** — Telegram pushes to your server's URL
- PythonAnywhere runs the **same Flask app** as an always-on worker
- Same code, same `.env` idea — different delivery

Today your bot stops needing you.

The front door: `api/index.py`

Route	Job
<code>/api/health</code>	"Is the worker alive?" → <code>OK</code>
<code>/api/webhook</code>	Where Telegram delivers messages
<code>/api/deploy</code>	Pulls new code on push (we'll wire this up)

- The webhook checks a **secret token** — forged requests get 403. The bot generates and registers it itself on first boot.
- `threaded=False` in `bot/clients.py` — handlers must finish in the request, or the platform may kill them mid-reply.

Deploy — one command

1. Sign up at pythonanywhere.com (free Beginner tier)
2. Create an API token: *Account* → *API token*
3. Add to your local `.env`:

```
PA_USERNAME=<your PA username>  
PA_API_TOKEN=<the token>
```

4. Ship it:

```
make deploy-pa
```

What just happened

The script created the web app, cloned **your fork**, built the venv, uploaded a production `.env` — including:

```
SQLITE_PATH=/home/<you>/bot.db  
WEBHOOK_URL=https://<you>.pythonanywhere.com/api/webhook
```

`WEBHOOK_URL` = auto-registration: every boot, the bot tells Telegram "*deliver here.*"
No manual `curl` needed.

Verify like an engineer

1. `https://<you>.pythonanywhere.com/api/health` → `OK`
2. Message your bot — the reply now comes from the server
3. **Close your laptop. Message it again.** 🎉
4. Something wrong? *Web tab* → *error log* — read the traceback

Push-to-deploy

One-time setup on your fork — *Settings* → *Secrets* → *Actions*:

Secret	Value
DEPLOY_SECRET	random string (also in PA <code>.env</code>)
PA_DEPLOY_URL	<code>https://<you>.pythonanywhere.com/api/deploy</code>

Then:

```
git commit -am "New personality" && git push
```

~3 seconds later the live bot is updated. This is how real teams ship.

The fine print

- PA free tier: click **"Run until 3 months from today"**-style renewal **every month** — or the app auto-disables
- PA emails you a week before. Set a phone reminder anyway.
- Free tier limits outbound internet to a whitelist — Telegram and Cerebras are on it

Final project — ship one original feature

- 🌐 Translator — Armenian ↔ English on demand
- ? Quiz master — asks, scores, keeps streaks (you have a store!)
- 😊😞 Mood reader — guesses the mood of a message
- 📝 Summarizer — paste a wall of text, get three lines

Or your own idea. It must be **live on your deployed bot** by demo time.

Demo day

1. Post your bot's username for the room
2. Try everyone's bot — find their feature
3. Vote: funniest persona, most useful feature, best surprise

Same bot as Tuesday morning — now it lives on the internet.

Next week

Your bot's brain is someone else's model behind an API.

Week 2: we train our own. 63 GB of Armenian text, one GPU, a 350-million-parameter model — built from scratch, fine-tuned to chat, and plugged into *this* bot.